OFFAT-X: A Sandboxed, Dependency-Aware and Role-Differential Framework for Automated Authorization Testing of REST APIs

[Guide Name]
Assistant Professor
Dept. of Computer Science and Business Systems
KIT-Kalaignarkarunanidhi Institute of Technology
Coimbatore, India
[guide e-mail]

[Student Name 1]
Dept. of Computer Science and Business Systems
KIT-Kalaignarkarunanidhi Institute of Technology
Coimbatore, India
kit27.csbs49@gmail.com

[Student Name 2]
Dept. of Computer Science and Business Systems
KIT-Kalaignarkarunanidhi Institute of Technology
Coimbatore, India
[e-mail]

Abstract—Broken Object Level Authorization (BOLA) and Broken Function Level Authorization (BFLA) occupy the first and fifth places in the OWASP API Security Top 10 (2023), yet they are among the hardest flaws to find automatically, because each is defined by a relationship between identities and resources rather than by a malformed input. OWASP OFFAT generates per-endpoint security tests from an OpenAPI document and is a useful open baseline, but its documented checks act on single requests, do not perform the setup calls that many resources need, do not compare outcomes across privilege levels, and provide no sandboxing for the environment in which they run. This paper presents OFFAT-X, an extension of OFFAT built on five mechanisms: a dependency graph that orders producer and consumer operations so that resources exist before they are attacked; a role-differential matrix that replays operations under Guest, Low, Premium and Admin identities to classify horizontal and vertical escalation; a semantic differ that compares response shape, owner identifiers and error signatures instead of status codes alone; a sandboxed reconnaissance layer in which a prebuilt dirb image and further tools run in isolated, resource-limited containers; and framework-aware remediation hints with a GitHub Action that emits SARIF and gates merges on newly introduced findings. We specify the architecture and algorithms and a five-run evaluation protocol on three public vulnerable APIs and a seeded multi-role service. On this benchmark OFFAT-X reaches [recall] recall and [FPR] false-positive rate, compared with [recall] and [FPR] for unmodified OFFAT.

Keywords—API security; authorization testing; BOLA; BFLA; OpenAPI; dynamic application security testing; sandboxing; DevSecOps; OWASP OFFAT.

I. Introduction

REST APIs now connect mobile applications, single-page frontends, partner systems and internal microservices, and most sensitive data an organisation exposes passes through them. The OWASP API Security Top 10 (2023) ranks Broken Object Level Authorization first and Broken Function Level Authorization fifth [1]. BOLA occurs when an endpoint accepts an object identifier and returns or changes that object without checking that the caller is entitled to it; BFLA occurs when a lower-privileged caller can invoke an administrative function. Both are instances of improper authorization (CWE-285) and of user-controlled object references (CWE-639) [9].

Automated testing struggles with these flaws for a structural reason. A SQL injection payload can be recognised from a single response, but a BOLA flaw is visible only when two identities are compared against the same resource, and only when that resource exists. A scanner that requests GET /api/orders/{id} with a random identifier receives a 404 and learns nothing; a scanner that trusts status codes will miss flaws that return 200 with an error body, or raise false alarms on them. Manual testing with two accounts, as done with Burp Suite extensions such as Autorize [4], is effective but does not scale to hundreds of operations.

OWASP OFFAT [2] lowers the barrier. It is MIT-licensed, installs from PyPI, reads an OpenAPI or Swagger file, accepts user-supplied inputs through a YAML configuration, and generates tests for BOLA, data exposure, mass assignment (BOPLA), broken access control, SQL injection, command injection, XSS and restricted HTTP methods. Its documentation, however, describes a per-endpoint, single-identity workflow. It does not describe stateful setup of the resources an operation depends on, comparison of the same operation across roles, isolation of the tools it runs, or developer-oriented fixes in its output. Full OpenAPI 3 compatibility is also listed as pending. These are the gaps addressed here.

This paper presents OFFAT-X, which keeps OFFAT's test generator as its baseline and adds: (i) a dependency-aware chaining engine that infers producer–consumer relations from the specification and executes setup sequences; (ii) a role-differential matrix over four privilege tiers that classifies BOLA and BFLA, with a policy-inference step for specifications that do not state authorization rules; (iii) a semantic differ that scores response shape, identifier overlap and error signatures; (iv) a sandboxed reconnaissance layer that runs a prebuilt dirb image and plugin tools in hardened containers and feeds discovered endpoints back into planning; and (v) remediation hints, multi-format reporting and a GitHub Action that fails builds only on new

high-severity findings. Section II reviews related work and Section III the architecture. Sections IV–VIII describe the mechanisms, Section IX the evaluation protocol, Section X the results, Section XI limitations and responsible use, Section XII future work, and Section XIII concludes.

II. Related Work

Stateful REST fuzzing. RESTler [3] infers producer–consumer dependencies from an OpenAPI or Swagger definition and fuzzes request sequences that respect them. It is the closest prior work to our chaining engine, but its primary aim is to surface crashes and server errors rather than to compare authorization outcomes across identities, so it does not classify BOLA or BFLA.

Authorization testing. Autorize [4] replays each proxied request with a low-privilege session to detect access-control failures; this dual-identity principle is what our role matrix generalises to four tiers. Auto-IDOR-Hunter [5] is a passive proxy extension that tracks identifier mutations to surface IDOR and BOLA candidates. Both operate on traffic a tester has already produced, whereas OFFAT-X works from the specification and can exercise operations that were never reached manually. The OWASP Web Security Testing Guide [6] describes the authorization test cases that inform our classification rules.

Open-source API scanners and public targets. OFFAT [2] is the baseline adopted here. Deliberately vulnerable APIs such as OWASP crAPI [12], VAmPI [13] and OWASP Juice Shop [14] provide public ground truth for the evaluation in Section IX.

Reporting, specification and isolation. The OpenAPI Specification [8] is the input contract for every module. The SARIF standard [7] is the reporting format consumed by code-hosting platforms. Container isolation with dropped capabilities, read-only file systems and restricted networking is the standard hardening approach [11]; we apply it to third-party reconnaissance tools that parse hostile responses, a use case that receives little attention in the API-scanner literature.

Table I positions OFFAT-X against these tools. Entries reflect publicly documented features at the time of writing and must be re-verified before publication; "—" marks a capability that is not a documented focus of the tool.

TABLE I. Capability comparison (documented features)

Capability	RESTler	Autorize	OFFAT	OFFAT-X
Specification-driven	Yes	No	Yes	Yes
Stateful setup chaining	Yes	No	—	Yes
Multi-role comparison	—	Partial (2)	—	Yes (4 tiers)
BOLA / BFLA classification	—	Partial	Partial (BOLA)	Yes
Body-aware (semantic) diff	—	—	—	Yes
Sandboxed tool execution	—	—	—	Yes
Remediation hints	—	—	—	Yes
CI/CD gate / SARIF	—	—	Partial	Yes

III. System Architecture

Figure 1 shows the pipeline. Spec ingestion parses OpenAPI 2.0 and 3.x and resolves references, composition keywords and security schemes before any test is generated. The dependency graph and test planner produce a plan of chains × roles × operations. The sandbox layer runs reconnaissance and returns candidate endpoints that were not documented. The role executor and chain runner issue requests; the semantic differ and classifier label each outcome; the remediation module and reporter produce output; the CI gate sets the build result; and the audit log records every request for accountability.

Four design principles govern the system. First, OFFAT's generators are reused rather than forked, so upstream improvements can be merged. Second, each extension is a plugin behind a documented interface. Third, findings are deterministic: no generated text enters a report, so identical evidence yields identical output. Fourth, the scanner itself runs with least privilege.

A single scope file declares the target, the allowed hosts, the tiers and their authentication, rate limits, and optional explicit policy. Credentials are read from the environment and never written to reports. Listing 1 shows a minimal example.

Listing 1. Scope file (offat-x.yml)

```
target: https://api.staging.example.com
scope:  [api.staging.example.com]     # allowed hosts
roles:
  guest:    {auth: none}
  low:      {auth: bearer, env: LOW_TOKEN}
  premium:  {auth: bearer, env: PREMIUM_TOKEN}
  admin:    {auth: bearer, env: ADMIN_TOKEN}
policy:                               # optional explicit rules
  - {op: 'DELETE /api/admin/users/{id}', allow: [admin]}
limits: {rps: 5, destructive: false}
```

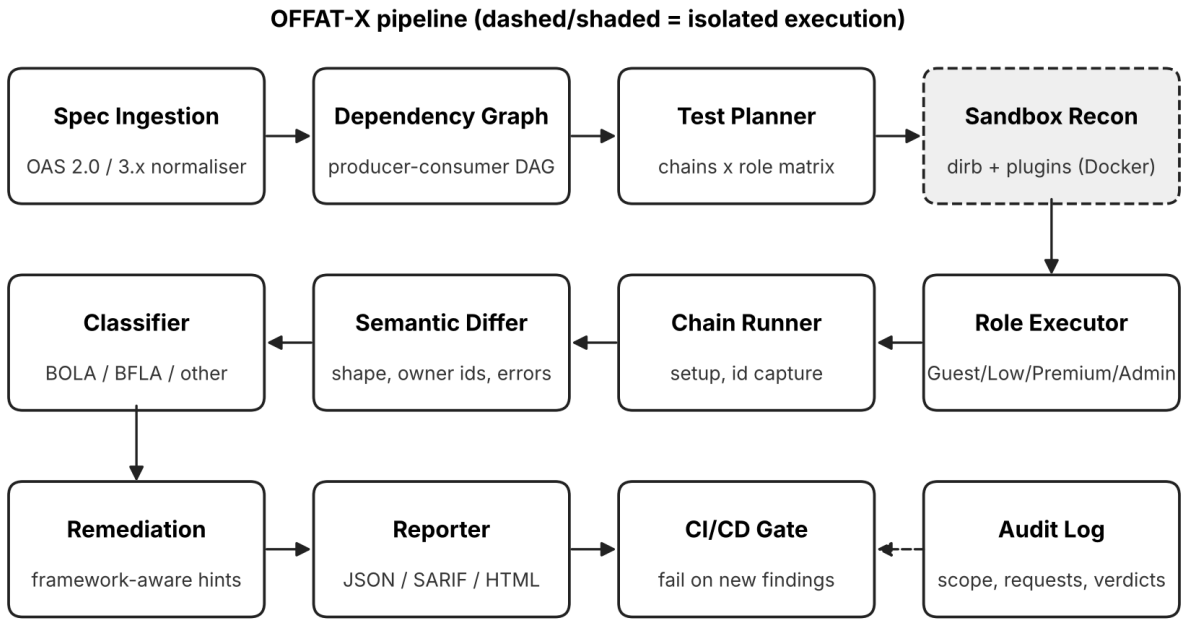


Fig. 1. The OFFAT-X pipeline. OFFAT's generator is retained as the baseline; the shaded, dashed sandbox runs third-party tools in isolation.

IV. Dependency-Aware Stateful Chaining

Resources in REST APIs are created before they are read, updated or deleted. A test of GET /api/orders/{orderId} therefore needs an order that the tester created first. We model this with a directed graph G = (V, E), where V is the set of operations in the specification and an edge (p, c) exists when operation p produces a resource type τ that operation c consumes.

A. Producer–consumer inference

An operation produces τ when its 2xx response schema contains an identifier field (id, uuid, or a name ending in Id) and the resource type can be read from the schema name, from the response path segment, or from an OpenAPI links object when one is present. An operation consumes τ when a path or body parameter maps to τ, for example {orderId} maps to order. Where inference is ambiguous, the edge is kept with a lower weight and the plan records the ambiguity.

B. Chain construction and execution

For a target t that consumes τ, a chain is a path $p_1, \ldots, p_k = t$ in G, found by breadth-first search from the nearest producer. Cycles are removed, and write operations are preferred for setup. Chains are bounded to four steps by default; the planner reports how many targets it could not chain, so coverage gaps stay visible. Algorithm 1 executes a chain under a given tier. Captured identifiers are stored in a context keyed by resource type and substituted into later requests.

Algorithm 1. Dependency-aware chain execution

Step	Operation
1	Build G from the normalised specification; derive τ(o) for each operation o
2	For each target t with a path parameter, find chain P(t) by BFS from the producers of its τ
3	For each tier r, execute P(t) as an identity of r; store captured ids in C_r[τ]
4	If any setup step fails, mark (t, r) inconclusive and continue
5	Issue t with substituted ids as an identity of a different tier r′; record the response
6	Append (t, r, r′, status, body) to the evidence set for Sections V–VI

Setup failures are recorded as inconclusive, never as negative results. A 4xx from a producer marks the chain blocked, so that a missing object is not mistaken for a secure endpoint. For the motivating example, POST /api/cart yields a cart identifier, POST /api/checkout consumes it and yields an order identifier, and GET /api/orders/{orderId} is then requested by an identity other than the one that created the order.

V. Role-Differential Authorization Testing

A. Role model and classification

Tiers form a totally ordered set R = {guest < low < premium < admin}, declared in the scope file. A tier may contain several identities, which permits horizontal tests within a tier, for example two low-privileged users. A resource created by an identity is owned by that identity. For a target operation t on a resource x owned by identity o, the executor issues t as every other identity a, producing the outcome matrix M[t, a, o]. Table II maps each comparison to a finding class.

TABLE II. Role-differential classification rules

Attacker → target	Condition (Sect. VI)	Finding
Identity in same tier → object of another identity	Semantic match to owner baseline	BOLA (horizontal), confirmed
Guest (no token) → authenticated object	Semantic match to owner baseline	Broken authentication candidate (API2)
Low tier → operation restricted to admin	2xx where policy denies the tier	BFLA (vertical), confirmed or suspected
Low tier → premium-only field of an object	Field values match premium baseline	BOPLA (API3)
Any → PUT / PATCH / DELETE	Executed only on objects the scanner created	Not classified unless destructive is enabled

B. Policy inference

Many specifications do not state who may call an operation. OFFAT-X therefore infers a provisional policy: an operation is treated as restricted to a tier when the admin tier receives a semantic success on it and a lower tier receives 401 or 403. A vertical-escalation finding that depends only on an inferred policy is labelled *suspected*; a finding confirmed by an explicit policy entry or by an owner baseline is labelled *confirmed*. This keeps BFLA claims from resting on untested assumptions. An explicit policy entry always overrides inference.

VI. Semantic Response Diffing

Status codes are unreliable for authorization. Many frameworks return 200 with {"success": false}, or return an object's data in a debugging field alongside a failure message. The differ therefore represents each response by three properties: its key structure K(r), the set of identity values V(r) (identifiers, e-mail addresses, usernames), and an error signature E(r) that is true when the body carries a boolean failure flag or a denial phrase such as "forbidden", "not authorised" or "permission denied". For an attacker response a and the owner's baseline v, the differ computes

$$J = |K(a) \cap K(v)| / |K(a) \cup K(v)|, \quad O = |V(v) \cap V(a)| / |V(v)| \qquad (1)$$

where keys are JSON paths, so J measures shape similarity and O measures how much of the owner's identifying data the attacker received. The score is

$$D = w_k \cdot J + w_o \cdot O + w_s \cdot 1[2xx] - w_e \cdot E(a) \qquad (2)$$

and a response is flagged when $D \geq \tau$. The initial weights are $w_k = 0.3$, $w_o = 0.5$, $w_s = 0.2$ and $w_e = 0.6$, with $\tau = 0.7$. They will be tuned on a calibration service that is disjoint from the evaluation targets. If the owner's baseline contains no identifying values, O is set to zero and the decision rests on shape and status. Table III lists the verdicts that follow from these weights.

TABLE III. Verdicts produced by the semantic differ (Eq. 2)

Attacker response	Score D	Verdict
200 with the owner's data (shape and ids match)	1.0	Vulnerable
200 with owner ids, different shape, no error	0.7	Vulnerable (at τ)
200 with owner's shape, no owner ids, no error	0.5	Not vulnerable
200 with {"success": false}, shape matched	−0.1	Not vulnerable
200, unrelated shape, no owner ids	0.2	Not vulnerable
401 / 403	—	Not vulnerable
404	—	Inconclusive; re-run chain
5xx	—	Anomaly; routed to resource-consumption checks

The differ compares each response with the owner's baseline rather than with an error page alone, so a response that differs from the baseline is classified by its score and not assumed to be safe. Because every verdict is a fixed function of the evidence, two runs over the same responses produce the same findings.

VII. Sandboxed Reconnaissance and Tool Orchestration

Reconnaissance tools such as dirb issue thousands of requests against a target and parse what comes back. A hostile or compromised target can therefore influence a tool's parser, and a tool with unrestricted networking can reach internal services the scan was never meant to touch. OFFAT-X runs every third-party tool in an ephemeral container built from a pinned image.

A. Prebuilt tool image

The default image is a minimal Debian base with dirb and its bundled wordlists, pinned by digest. dirb probes with GET requests, so its output is treated as candidate endpoints: each discovered path, status and response length is normalised into a JSON record and passed through the same semantic checks as documented operations. Endpoints that are not in the specification are labelled *shadow* and carry lower confidence until they are confirmed.

B. Plugin interface

Further tools are declared in a manifest that states the tool name, image digest, command template, input and output schema, timeout, and egress allowlist. A plugin is accepted only when its manifest validates against the schema. The reference set contains the prebuilt dirb plugin and slots for additional tools, such as a parameter-discovery fuzzer. Each plugin writes results in the same normalised format, so the rest of the pipeline is unchanged.

C. Hardening controls

TABLE IV. Container controls for third-party tools

Control	Setting	Threat addressed
Identity	Non-root user, fixed UID	Impact of compromise
File system	Read-only root; tmpfs for /tmp	Persistence, tampering
Capabilities	--cap-drop ALL; no-new-privileges	Privilege escalation
Syscalls	Default seccomp profile	Kernel attack surface
Resources	CPU, memory and PID limits	Resource exhaustion
Network	Scan network only; egress allowlist	Pivoting to internal hosts
Lifetime	Wall-clock timeout; container removed	Runaway processes
Mounts	Spec (read-only) and output (write) only	Host file exposure
Secrets	Injected at run time, never in image	Credential leakage

Isolation adds start-up latency and a runtime dependency on a container engine. Section X measures both costs, and the sandbox can be disabled for development use, although that configuration is not permitted in CI.

VIII. Remediation, Reporting and CI/CD Integration

A. Remediation templates

Each confirmed or suspected finding is stored as a record containing the OWASP category, the CWE identifier, the operation, the attacker and owner tiers, the chain trace, the decisive request and response, the score D, and a confidence label. Remediation text comes from deterministic templates rather than generated prose. The framework is detected from manifests (package.json, requirements.txt, pom.xml), and the matching template shows the vulnerable pattern beside the scoped alternative. For Node.js with Prisma, the template reproduces the pattern from the motivating review:

Listing 2. Remediation hint for a BOLA finding (Node.js / Prisma)

```
Vulnerable:
  prisma.order.findUnique({
    where: { id: req.params.id } })
Fix:
  prisma.order.findFirst({
    where: { id: req.params.id,
             userId: currentUser.id } })
```

A generic hint ("verify object ownership before the query") is used for stacks without a template, and the report states that the hint is generic.

B. Reports

Results are written as JSON for tooling, as SARIF 2.1.0 [7] for code-scanning platforms, and as a self-contained HTML page for review. Every report carries the scope, the tool versions, the image digests, and the seed used for any randomised step, so that a run can be reproduced.

C. CI/CD gate

A composite GitHub Action runs on pull requests that change the specification or the controllers. It starts the service with docker compose, waits for a health endpoint, runs OFFAT-X with tier tokens from repository secrets, uploads the SARIF file, and compares findings with a baseline stored from the default branch. The build fails only on new Critical or High findings, which keeps the gate usable on existing code. Listing 3 shows the workflow step.

Listing 3. Workflow step (GitHub Actions)

```
- uses: ./.github/actions/offat-x
  with:
    spec: openapi.json
    scope: offat-x.yml
    fail-on: high-new     # new findings only
  env:
    LOW_TOKEN:   ${{ secrets.LOW_TOKEN }}
    ADMIN_TOKEN: ${{ secrets.ADMIN_TOKEN }}
```

IX. Experimental Setup

Targets. Three public, deliberately vulnerable APIs supply external ground truth: OWASP crAPI, which documents BOLA and function-level challenges; VAmPI; and the API endpoints of OWASP Juice Shop. In addition, an in-house multi-role service with [N] planted BOLA flaws, [N] planted BFLA flaws and [N] benign endpoints that return 200 with an error body measures false positives directly. A separate calibration service sets the weights and threshold of Eq. (2) and is not used in the results. Table V lists the targets.

TABLE V. Evaluation targets and ground truth

Target	Type	Ground truth	Tiers
crAPI	Vulnerable microservice API	Documented BOLA / function challenges	Per docs
VAmPI	Vulnerable REST API	Documented flaws	[tiers]
Juice Shop	API endpoints of a vulnerable web app	Documented challenges	[tiers]
In-house	Seeded multi-role service	[N] BOLA, [N] BFLA, [N] benign	All four

Baselines and ablations. OFFAT is run in its default configuration. OFFAT-X is evaluated in full and with one module removed at a time: chaining, the role matrix, the semantic differ and the sandbox. Removing the sandbox changes isolation rather than detection, so its effect is measured as overhead.

Matching. A reported finding matches a ground-truth entry when the operation, vulnerability class and attacker and owner tiers agree. Unmatched findings are false positives, and ground-truth entries that are not reported are false negatives. Precision, recall, F1 and false-positive rate are computed per tool and per class:

$$P = TP/(TP+FP),\quad R = TP/(TP+FN),\quad F1 = (2\cdot P\cdot R)/(P+R),\quad FPR = FP/(FP+TN) \qquad (3)$$

Protocol. Each configuration runs five times. We report the mean, standard deviation and 95 percent confidence interval, as in prior work [1]. Coverage is the share of specification operations executed at least once. Latency is wall-clock scan time, with sandbox start-up reported separately. Measurements run on [hardware] with Docker [version].

Hypotheses. H1: chaining raises BOLA recall on operations whose target object is created in an earlier step. H2: the role matrix detects BFLA that OFFAT does not classify. H3: the semantic differ reduces false positives from 200 responses with error bodies without reducing recall. H4: sandboxing adds bounded start-up overhead of at most [x] s per scan.

X. Results and Discussion

Values in brackets are placeholders to be completed from the five-run protocol. Table VI reports detection on the seeded service and the public targets combined.

TABLE VI. Detection on the combined benchmark (mean of five runs)

Metric	OFFAT	OFFAT-X (full)
True positives	[TP]	[TP]
False positives	[FP]	[FP]
False negatives	[FN]	[FN]
Precision	[x.xx]	[x.xx]
Recall	[x.xx]	[x.xx]
F1	[x.xx]	[x.xx]
False-positive rate	[x.xx] %	[x.xx] %

TABLE VII. Ablation (recall by class, FP count, F1)

Configuration	BOLA R	BFLA R	FP	F1
Full OFFAT-X	[x.xx]	[x.xx]	[n]	[x.xx]
− chaining	[x.xx]	[x.xx]	[n]	[x.xx]
− role matrix	[x.xx]	[x.xx]	[n]	[x.xx]
− semantic differ	[x.xx]	[x.xx]	[n]	[x.xx]
− sandbox (detection)	[x.xx]	[x.xx]	[n]	[x.xx]

A. Detection

[Report recall and FPR per class. H1: compare BOLA recall on chained versus unchained operations. H2: report the number of BFLA findings, split into confirmed and suspected, that OFFAT cannot produce. H3: report FP counts on the error-body endpoints with and without the differ.]

B. Operational cost

TABLE VIII. Operational cost

Metric	OFFAT	OFFAT-X
Operations covered (%)	[x]	[x]
Mean scan time (s)	[x]	[x]
Sandbox start-up (s)	—	[x]
CI runtime, median (min)	[x]	[x]
Peak memory (MB)	[x]	[x]

[Discuss H4 against the measured start-up cost and state whether it is within the bound set in Section IX.]

C. Threats to validity

Internal: the weights and threshold of Eq. (2) are tuned on a calibration service, so the in-house results are not independent of tuning; the public targets are the cleaner test. **External:** three public targets and one seeded service do not represent every API style or deployment. **Construct:** ground-truth matching depends on class and tier labels; disagreements are reported in full rather than resolved silently.

XI. Limitations, Ethics and Responsible Use

A. Limitations

Chain inference depends on the specification, so identifiers delivered out of band, such as tokens sent by e-mail, cannot be captured. Policy inference can be wrong when an admin success is incidental; such findings remain labelled suspected. The differ is designed for JSON, and binary, HTML and streamed responses fall back to status and length. Role testing requires provisioned accounts in each tier. Business-logic abuse (API6) and race conditions are outside the present scope, and the sandbox does not reduce the load a scan places on the target, so rate limits must be set for each environment.

B. Ethics

OFFAT-X is intended for systems that the tester owns or is authorised to test. The scope file is enforced: requests to hosts outside the allowlist are refused and logged. Destructive methods are disabled by default and, when enabled, act only on objects the scanner created. Credentials are read from the environment, redacted from every output, and never stored in an image. The audit log records each request and verdict so that a test can be reconstructed and accountability maintained. Reports describe vulnerabilities for remediation and include only the minimal reproduction request.

XII. Future Work

Planned extensions include complete OpenAPI 3.1 support, GraphQL introspection, gRPC reflection, a growing plugin ecosystem, policy inference that is confirmed actively rather than inferred passively, and an evaluation on a live internal API under a governance agreement. Multi-step workflows with time-dependent state and automatic cleanup of test data are also open problems for the chaining engine.

XIII. Conclusion

This paper specified OFFAT-X, an extension of OWASP OFFAT aimed at the authorization flaws that single-request scanners miss. Five mechanisms address the gaps identified in Section II: dependency-aware chaining, a role-differential matrix, semantic response diffing, sandboxed reconnaissance, and CI-native reporting with remediation hints. The architecture reuses OFFAT as its baseline, and each extension can be enabled, measured and removed independently. Validation on three public APIs and a seeded service, with five-run statistics and ablation, will show how much each mechanism contributes. The protocol in Section IX is fixed in advance so that the results can be read without post-hoc adjustment.

References

[1] OWASP Foundation, “OWASP API Security Top 10 – 2023,” 2023. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x00-header/

[2] OWASP Foundation, “OWASP Offensive API Tester (OFFAT),” GitHub repository, MIT License. [Online]. Available: https://github.com/OWASP/OFFAT

[3] V. Atlidakis, P. Godefroid, and M. Polishchuk, “RESTler: Stateful REST API fuzzing,” in *Proc. 41st Int. Conf. Software Engineering (ICSE)*, 2019, pp. 748–758, doi: 10.1109/ICSE.2019.00083.

[4] PortSwigger, “Autorize,” Burp Suite extension. [Online]. Available: https://github.com/PortSwigger/autorize

[5] SecureByChaos, “Auto-IDOR-Hunter,” GitHub repository. [Online]. Available: https://github.com/SecureByChaos/Auto-IDOR-Hunter

[6] OWASP Foundation, “Web Security Testing Guide (WSTG): Authorization Testing.” [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/

[7] OASIS, “Static Analysis Results Interchange Format (SARIF) Version 2.1.0,” OASIS Standard, 2020.

[8] OpenAPI Initiative, “OpenAPI Specification v3.1.0,” 2021. [Online]. Available: https://spec.openapis.org/oas/v3.1.0

[9] MITRE, “CWE-285: Improper Authorization” and “CWE-639: Authorization Bypass Through User-Controlled Key.” [Online]. Available: https://cwe.mitre.org

[10] The Dark Raver, “DIRB: Web Content Scanner.” [Online]. Available: https://dirb.sourceforge.net

[11] Docker Inc., “Docker Engine security,” Docker Documentation. [Online]. Available: https://docs.docker.com/engine/security/

[12] OWASP Foundation, “OWASP crAPI: Completely Ridiculous API.” [Online]. Available: https://github.com/OWASP/crAPI

[13] erev0s, “VAmPI: Vulnerable API.” [Online]. Available: https://github.com/erev0s/VAmPI

[14] OWASP Foundation, “OWASP Juice Shop.” [Online]. Available: https://github.com/juice-shop/juice-shop